

Lista2 – pilhas, filas e listas

```
#include <stdio.h>
#include <stdlib.h>
typedef struct no{
    int valor;
    struct no *proximo;
}No;
typedef struct{
    No *inicio;
}Lista;
//-----
void inserirMeio(Lista *lista,int num,int ant){
    No *aux, *novo = malloc(sizeof(No));
    if(novo){
        novo->valor = num;
        if(lista->inicio == NULL){
            novo->proximo = NULL;
            lista->inicio = novo;
        }else{
            aux = lista->inicio;
            while(aux->valor != ant && aux->proximo)
                aux = aux->proximo;
            novo->proximo = aux->proximo;
            aux->proximo = novo;
        }
    }else printf("Erro ao localizar memoria.\n");
}
```

```
void imprimir(Lista lista){
    No *no = lista.inicio;
    while(no){
        printf("%d ", no->valor);
        no = no->proximo;
    }
}
//-----
int main(void) {
    int opcao, valor, anterior;
    No *no;
    Lista lista;
    lista.inicio=NULL;
    inserirMeio(&lista, 10, 15);
    inserirMeio(&lista, 20, 0);
    inserirMeio(&lista, 30, 10);
    inserirMeio(&lista, 40, 20);
    inserirMeio(&lista, 50, 40);
    imprimir(lista);
    system("pause");
    return 0;
}
```

- 1) Com base no código fonte acima escrito em linguagem C, apresente exatamente o resultado que será apresentado ao usuário após a execução completa do programa.
- 2) Utilizando linguagem C, apresente apenas a estrutura mínima do nó de uma lista circular.

Lista2 – pilhas, filas e listas

3) Analise o código abaixo e implemente uma função chamada **sair()** que seja responsável por liberar corretamente toda a memória alocada dinamicamente antes da finalização do programa.

```

1 #include <stdlib.h>
2 #include <stdio.h>
3
4 typedef struct apelido_no{
5     int dado;
6     struct apelido_no *proximo;
7 }no;
8 no *top=NULL;
9 //-----
10 int entrada_dados(){
11     int valor;
12     printf("\nValor a empilhar: ");
13     scanf("%d",&valor);
14     return valor;
15 }
16 //-----
17 void push(int item){
18     no *novo=malloc(sizeof(no));
19     //verificar se há memória
20     novo->dado=item;
21     novo->proximo=top;
22     top=novo;
23     system("pause");
24 }
25 //-----
26 void pop(){
27     if (top==NULL)
28         printf("A pilha esta vazia\n");
29     else{
30         no *temp;

```

```

31         temp=top;
32         top=top->proximo;
33         free(temp);
34     }
35     system("pause");
36 }
37 //-----
38 int main(){
39     int n,opcao;
40     do{
41         system("cls");
42         printf("\nMenu\n1. Empilha");
43         printf("\n2. Desempilha\n3. Sair");
44         printf("\nOpcao (0-3):");
45         scanf("%d",&opcao);
46         switch (opcao){
47             case 1:
48                 n=entrada_dados();
49                 push(n); //empilhar
50                 break;
51             case 2:
52                 pop(); //desempilhar
53                 break;
54             case 3:
55                 //sair();
56                 break;
57         }
58     }while (opcao!=3);
59     system("pause");
60     return 0;

```

4) Considerando as estruturas e a função **buscar** abaixo, explique a lógica implementada na linha 5 da função para uma busca sequencial em uma lista simplesmente encadeada.

```

typedef struct no{
    int valor;
    struct no *proximo;
}No;

typedef struct{
    No *inicio;
    int tam;
}Lista;

```

```

1 No* buscar(Lista *lista, int num){
2     No *aux, *no = NULL;
3
4     aux = lista->inicio;
5     while (aux && aux->valor != num)
6         aux = aux->proximo;
7     if (aux)
8         no = aux;
9     return no;
10 }

```

Lista2 – pilhas, filas e listas

5) Uma lista simplesmente encadeada é uma estrutura de dados em que cada elemento, chamado de nó, contém um valor e uma referência para o próximo nó na sequência. Os nós são encadeados um após o outro, formando uma lista linear. O último nó da lista tem sua referência de próximo como nula, indicando o final da lista. Uma lista simplesmente encadeada oferece uma maneira eficiente de inserir e remover elementos no início da lista, já que apenas os ponteiros precisam ser atualizados. No entanto, o acesso direto a um elemento em uma posição específica requer percorrer a lista a partir do primeiro nó.

Diante do contexto apresentado, considere uma lista simplesmente encadeada que contém os seguintes elementos: 10 → 20 → 30 → 40 → 50 e apresente a sequência da lista após a lista receber NULL e a remoção do elemento 30.

6) Apresente as características acerca de Listas Circulares, bem como, um modelo prático do nosso dia a dia.

7) Analise o código abaixo desenvolvido em linguagem C e **APRESENTE** a representação gráfica (desenhando a estrutura) após a execução completa do programa, inclusive sinalizando onde a variável ***p** estará apontando.

```

1 #include <stdlib.h>
2
3 typedef struct no{
4     int dado;
5     struct no *proximo;
6 }NO;
7
8 NO *p=NULL;
9 //-----
10 void acao_1(int v){
11     NO *novo=malloc(sizeof(NO));
12     novo->dado=v;
13     novo->proximo=p;
14     p=novo;
15 }
16 //-----
17 void acao_2(){
18     if (p==NULL)
19         printf("pilha vazia\n");
20     else{
21         NO *temp;
22         temp=p;
23         p=p->proximo;
24         free(temp);
25     }
26 }
  
```

```

27 //-----
28 int main(){
29     int n,opcao;
30     system("cls");
31     acao_1(71);
32     acao_1(8);
33     acao_1(15);
34     acao_2();
35     acao_1(19);
36     system("pause");
37     return 0;
38 }
  
```

Lista2 – pilhas, filas e listas

8) Uma pilha é uma estrutura de dados fundamental em ciência da computação. Ela segue uma política de acesso aos elementos que permite adicionar e remover elementos de uma extremidade específica.

Observando as afirmações abaixo, assinale qual das seguintes alternativas abaixo descreve corretamente a principal característica de uma pilha:

- a)** Em uma pilha, os elementos são inseridos e removidos de qualquer posição.
- b)** Uma pilha é uma estrutura de dados em que o primeiro elemento inserido é o primeiro a ser removido (FIFO).
- c)** Em uma pilha, os elementos podem ser removidos de qualquer posição, mas só podem ser inseridos no topo.
- d)** Uma pilha é uma estrutura de dados que segue o princípio de último a entrar, primeiro a sair (LIFO).
- e)** Em uma pilha, a remoção de elementos só é permitida no final, enquanto a inserção é permitida em qualquer posição.

9) Na abordagem da estrutura de dados do tipo lista, existe uma diversidade, que pode ser implementada de três formas, simplesmente encadeada, duplamente encadeada e circular. Partindo da implementação de uma lista simplesmente encadeada, assinale a alternativa que corresponde a estrutura de um nó para uma lista simplesmente encadeada.

- a)** Percorre a lista nos dois sentidos, do início para o fim e do fim para o início.
- b)** Cada nó possui dois ponteiros, um para o próximo nó e um para o nó anterior.
- c)** Tem como característica principal que o último nó da lista aponta para o primeiro nó da lista.
- d)** Na implementação, o programador deve ter o cuidado para que a estrutura não entre um loop infinito.
- e)** Possui um nó inicial e a partir dele é possível percorrer toda a lista em uma única direção, do início para o fim.

Lista2 – pilhas, filas e listas

10) A estrutura de dados fila é amplamente utilizada em sistemas computacionais para organizar elementos de forma ordenada. Considerando o funcionamento de uma fila, assinale a alternativa correta:

- a) A inserção e a remoção ocorrem sempre no topo da estrutura.
- b) A fila segue o princípio LIFO (*Last In, First Out*).
- c) O primeiro elemento inserido é o primeiro elemento removido da fila.
- d) Os elementos podem ser removidos de qualquer posição da fila.
- e) A fila permite inserções apenas no início da estrutura.

11) As listas duplamente encadeadas possuem características específicas que as diferenciam das listas simplesmente encadeadas. Assinale a alternativa correta:

- a) Cada nó possui apenas um ponteiro para o próximo elemento.
- b) O último nó aponta obrigatoriamente para o primeiro nó.
- c) A lista permite percorrer os elementos apenas do início para o fim.
- d) Cada nó possui referência para o próximo e para o elemento anterior.
- e) Os elementos não podem ser removidos da estrutura.

12) Em uma pilha, as operações básicas recebem nomes específicos. Assinale a alternativa que apresenta corretamente a operação responsável por remover um elemento do topo da pilha:

- a) Enqueue
- b) Insert
- c) Pop
- d) Push
- e) Peek

Lista2 – pilhas, filas e listas

13) As filas circulares são utilizadas para otimizar o aproveitamento de memória em determinadas aplicações. Sobre filas circulares, assinale a alternativa correta:

- a) O último elemento inserido nunca poderá ser removido.
- b) O final da fila pode se conectar novamente ao início da estrutura.
- c) A fila circular permite apenas uma inserção de dados.
- d) Os elementos são acessados aleatoriamente.
- e) A fila circular elimina completamente o conceito FIFO.

14) As listas simplesmente encadeadas apresentam vantagens e limitações. Assinale a alternativa que representa corretamente uma característica dessa estrutura:

- a) Cada nó possui ponteiro para o próximo e para o anterior.
- b) A lista pode ser percorrida apenas em uma direção.
- c) O último nó aponta obrigatoriamente para o primeiro.
- d) Os elementos só podem ser inseridos no final da lista.
- e) A estrutura utiliza obrigatoriamente memória sequencial.

15) Em relação às pilhas, assinale a alternativa que apresenta uma aplicação prática dessa estrutura de dados:

- a) Controle de impressão em uma impressora.
- b) Gerenciamento de processos paralelos.
- c) Controle de chamadas telefônicas.
- d) Histórico de navegação de páginas em navegadores.
- e) Armazenamento permanente em banco de dados.

16) As filas são frequentemente utilizadas em sistemas operacionais. Assinale a alternativa que melhor representa uma aplicação típica de filas:

- a) Controle de recursividade em funções.
- b) Processamento de impressão de documentos.
- c) Armazenamento de variáveis locais.
- d) Implementação de árvores binárias.
- e) Controle de memória cache LIFO.

Lista2 – pilhas, filas e listas

17) Em uma lista circular, existe uma característica importante relacionada ao último elemento da estrutura. Assinale a alternativa correta:

- a)** O último nó não possui ligação com nenhum elemento.
 - b)** O último nó aponta para o nó anterior.
 - c)** O último nó aponta para o primeiro nó da lista.
 - d)** O último nó armazena obrigatoriamente valor nulo.
 - e)** A lista circular não possui nó inicial.
-

18) Considerando as operações realizadas em pilhas, assinale a alternativa que descreve corretamente a operação Push:

- a)** Remove o elemento do topo da pilha.
 - b)** Consulta o elemento da base da pilha.
 - c)** Insere um novo elemento no topo da pilha.
 - d)** Remove todos os elementos da pilha.
 - e)** Ordena os elementos da pilha automaticamente.
-

19) As estruturas de dados do tipo lista permitem diferentes formas de implementação. Sobre listas encadeadas, assinale a alternativa correta:

- a)** Os elementos precisam estar armazenados em posições sequenciais de memória.
- b)** Cada elemento da lista é chamado de nó.
- c)** Listas encadeadas não permitem inserção dinâmica de elementos.
- d)** O acesso aos elementos ocorre exclusivamente de trás para frente.
- e)** As listas encadeadas não utilizam ponteiros.

Lista2 – pilhas, filas e listas**20) Exercício – Integração de Lista, Fila e Pilha**

As turmas de Análise e Desenvolvimento de Sistemas (ADS) e Engenharia de Software (ESOFT) da Unicesumar estão realizando uma promoção para arrecadar recursos para a formatura.

Para a retirada das pizzas vendidas, foi realizado um sorteio com 10 participantes. Os nomes dos sorteados deverão ser armazenados em uma **LISTA**, que representará o cadastro das pessoas autorizadas a retirar as pizzas.

No dia da retirada, os participantes chegarão à pizzaria e serão atendidos por ordem de chegada. Portanto, deverá ser utilizada uma **FILA** para organizar os sorteados que aguardam atendimento.

A pizzaria produzirá as pizzas conforme sua disponibilidade. Cada pizza pronta será embalada e colocada sobre as demais, formando uma **PILHA** de pizzas prontas para entrega.

O funcionamento do sistema deverá seguir as seguintes regras:

- Cadastrar os 10 sorteados em uma LISTA.
- À medida que os sorteados chegarem à pizzaria, inseri-los na FILA de atendimento.
- Conforme as pizzas forem ficando prontas, elas deverão ser EMPILHADAS na PILHA de pizzas prontas.
- Quando ocorrer uma entrega:
 1. A pessoa que está no início da FILA deverá ser atendida (desenfileiramento);
 2. A pizza que estiver no topo da PILHA deverá ser retirada (desempilhamento);
 3. Na LISTA de sorteados, a pessoa deverá ser marcada como "Pizza Retirada".
- O processo continua até que todas as pizzas tenham sido entregues.

O programa deverá permitir:

- Inserir e visualizar os sorteados da LISTA;
- Inserir e visualizar pessoas na FILA de atendimento;
- Inserir e visualizar pizzas na PILHA de pizzas prontas;
- Realizar a entrega de pizzas, efetuando simultaneamente o desenfileiramento da pessoa e o desempilhamento da pizza;
- Consultar a LISTA de sorteados, identificando quais pessoas já retiraram sua pizza e quais ainda não retiraram.

Observações

- As estruturas de dados (LISTA, FILA e PILHA) deverão ser implementadas utilizando linguagem C.
- Cada aluno deverá definir livremente as *structs* necessárias, bem como seus atributos, de acordo com sua interpretação da regra de negócio.
- É recomendado que as *structs* possuam informações suficientes para identificar pessoas, pizzas e o status de retirada.
- A implementação deverá demonstrar claramente o uso correto das operações fundamentais de cada estrutura de dados:
 - Lista: inserção, consulta e atualização;
 - Fila: *enqueue* (enfileirar) e *dequeue* (desenfileirar);
 - Pilha: *push* (empilhar) e *pop* (desempilhar).